

DataPortal Documentation Center

[Documentation Center](#) > [Platform](#) > [Application management](#) > [Services](#) > [GKE](#) >

[Join an existing cluster](#) >

Join an existing cluster

New application teams can onboard services onto DevEx Multi-Tenant Kubernetes Clusters (Shared GKE Clusters), hosted on the Google Cloud Platform (GCP) Google Kubernetes Engine (GKE).

This guide explains:

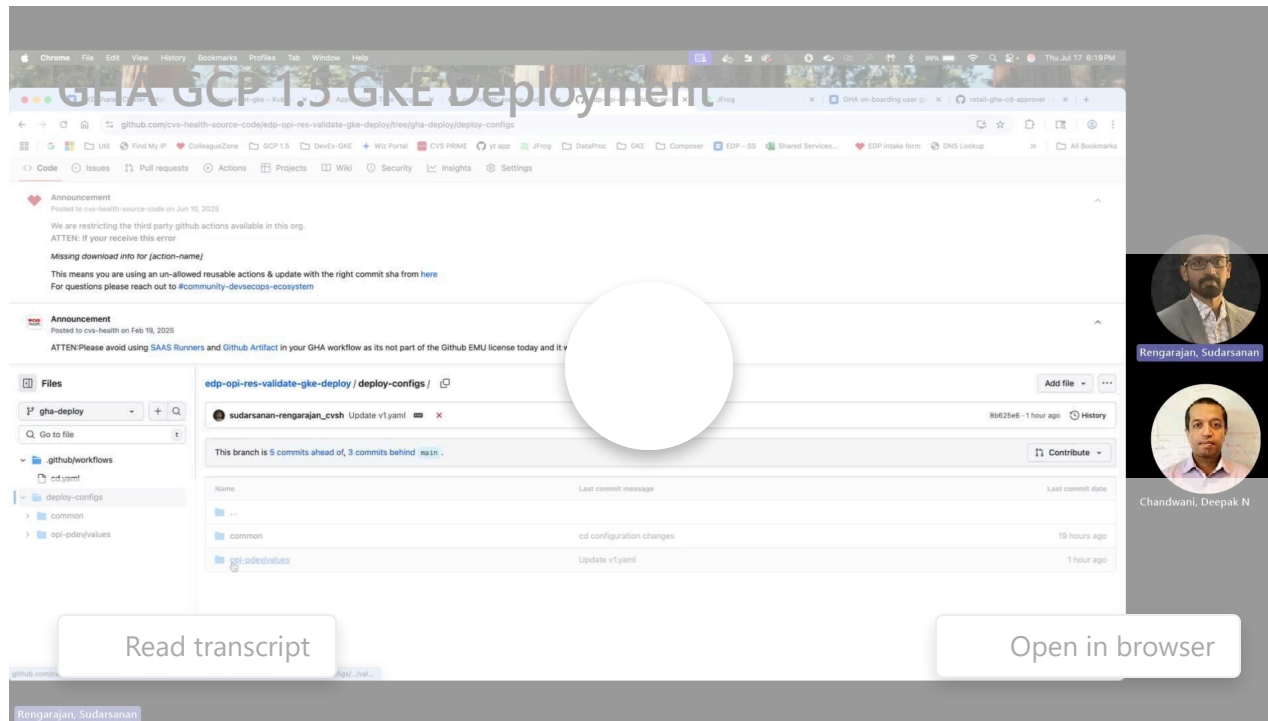
1. Which shared cluster to join.
2. DevEx CI/CD pipelines via GitHub Actions (GHA).
3. Configuration standards and GitHub repo structure.

You can:

- Deploy the GKE with minimal manual intervention.
- Manage and maintain your manifests repos in a structured, scalable manner.
- Leverage DevEx tools and automation.

Video introduction

The following video provides a demonstration of how to join an existing GKE cluster.



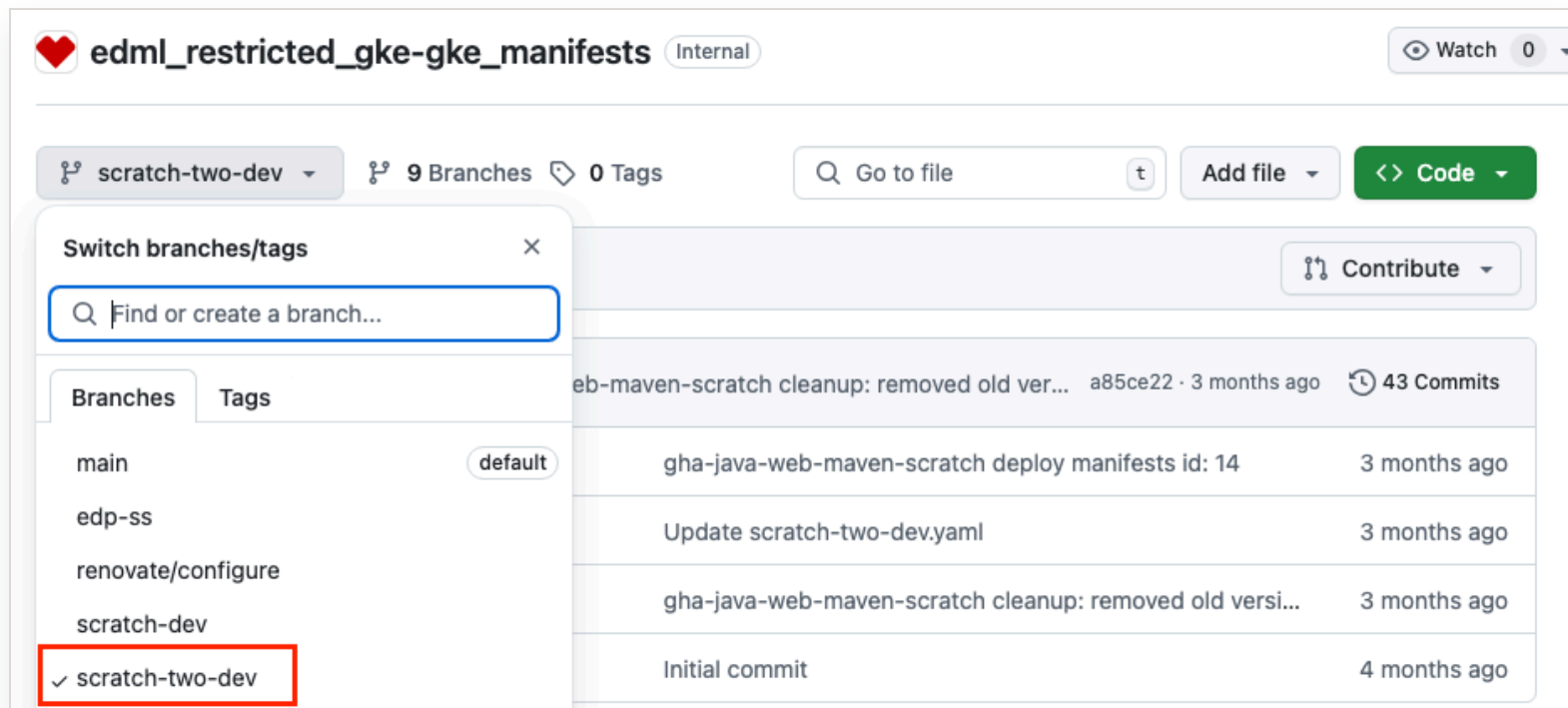
Manifest repositories

Each cluster has a manifest repository with the deployment, namespace, and service configurations for each line of business (LOB) and application on the cluster. You must create a single branch for your application in the manifest repo to avoid entering an intake form to the shared services team for a new branch.

From [Confluence > GKE Shared Cluster Details](#) [↗](#), the **Manifest Repo** column shows the manifest repos for each LOB.

Example manifest repo

Each branch in the manifest repository is a namespace for the corresponding application in the corresponding cluster. For example, from the **edml_restricted_gke-gke_manifests** repository, the [scratch-two-dev](#) [↗](#) branch provides an example manifest repo with recommended folder structure.

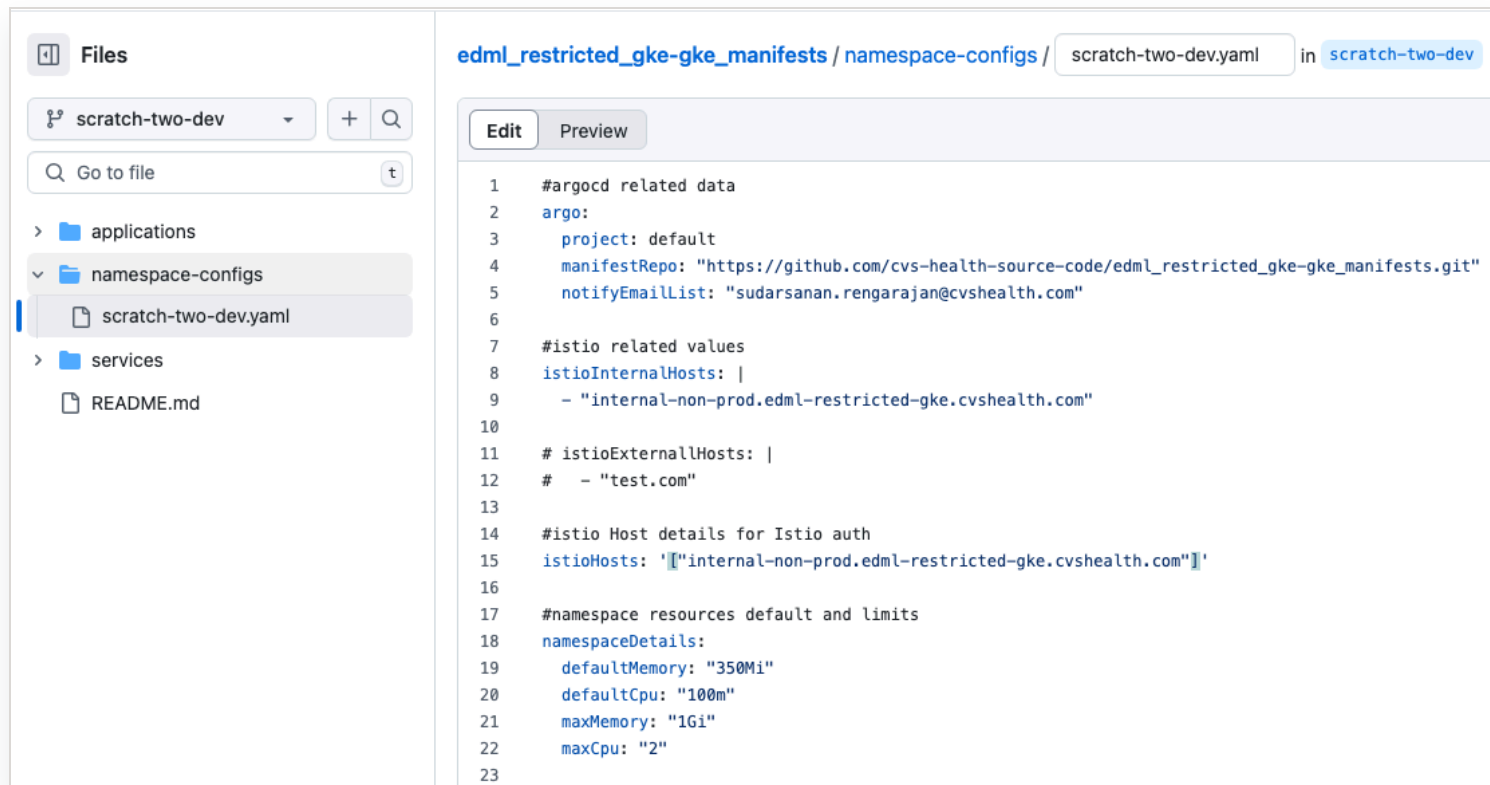


The **namespace-configs** folder provides the manifest for your namespace, including a yaml file with the namespace name (e.g., **scratch-two-dev.yaml**).

Create your namespace

From the manifest repository for your cluster, you must create a branch for your application to create a namespace and join the cluster.

1. From the **Manifest Repo** column of [Confluence > GKE Shared Cluster Details](#) [↗](#), find the manifest repo for the cluster for your LOB.
2. If the manifest repo does not already include a branch for your application, create one. For more: [GitHub Docs > Creating a branch](#) [↗](#).
3. From the **scratch-two-dev.yaml** file of the **edml_restricted_gke-gke_manifests** repository [scratch-two-dev](#) [↗](#) branch, copy the code.



The screenshot shows a GitHub repository interface. On the left, the 'Files' sidebar displays the directory structure: 'scratch-two-dev' (selected), 'applications', 'namespace-configs' (expanded), 'scratch-two-dev.yaml' (selected), 'services', and 'README.md'. The main area shows the content of 'scratch-two-dev.yaml' in the 'namespace-configs' directory. The file content is as follows:

```
1 #argocd related data
2 argo:
3   project: default
4   manifestRepo: "https://github.com/cvs-health-source-code/edml_restricted_gke-gke_manifests.git"
5   notifyEmailList: "sudarsanan.rengarajan@cvshealth.com"
6
7 #istio related values
8 istioInternalHosts: |
9   - "internal-non-prod.edml-restricted-gke.cvshealth.com"
10
11 # istioExternalHosts: |
12 #   - "test.com"
13
14 #istio Host details for Istio auth
15 istioHosts: ["internal-non-prod.edml-restricted-gke.cvshealth.com"]
16
17 #namespace resources default and limits
18 namespaceDetails:
19   defaultMemory: "350Mi"
20   defaultCpu: "100m"
21   maxMemory: "1Gi"
22   maxCpu: "2"
23
```

4. Paste the code into the namespace yaml file for your application.
5. From the **Default Endpoint** column of [Confluence > GKE Shared Cluster Details](#) [↗](#), copy your **AppURL**.
6. From the **istioInternalHosts** and **istioHosts** properties of the namespace configuration file, paste your **AppURL** (e.g., **"edp-ss.dev.cvshealth.com"**).

The screenshot shows a file editor interface with a sidebar on the left displaying a file tree. The main area shows the content of the file 'scratch-two-dev.yaml' under the path 'edml_restricted_gke-gke_manifests / namespace-configs / scratch-two-dev.yaml'. The file is owned by 'sudarsanan-rengarajan_cvsh' and was last updated. The code is displayed in a light blue theme with line numbers. The following YAML content is shown:

```

1  #argocd related data
2  argo:
3    project: default
4    manifestRepo: "https://github.com/cvs-health-source-code/edml_restricted_gke-gke_manifests.git"
5    notifyEmailList: "sudarsanan.rengarajan@cvshealth.com"
6
7  #istio related values
8  istioInternalHosts: |
9    - "internal-non-prod.edml-restricted-gke.cvshealth.com"
10   - "edp-ss.dev.cvshealth.com"
11
12  # istioExternalHosts: |
13  #   - "test.com"
14
15  #istio Host details for Istio auth
16  istioHosts: '["internal-non-prod.edml-restricted-gke.cvshealth.com","edp-ss.dev.cvshealth.com"]'
17
18  #namespace resources default and limits
19  namespaceDetails:
20    defaultMemory: "350Mi"
21    defaultCpu: "100m"
22    maxMemory: "1Gi"
23    maxCpu: "2"

```

7. From ColleagueZone, fill out the [Domain Name System \(DNS\) Request](#) [↗](#) form and click **Order Now**.

From the **DNS Destination** field, enter the IP address from your cluster's **Istio Primary IP** column from [Confluence > GKE Shared Cluster Details](#) [↗](#) (e.g., **10.112.152.2**).

DNS Request Details

Action	DNS Record Type	DNS Source	DNS Destination
Add	A: Address Record	edp-ss.dev.cvshealth.com	10.112.152.2

Close

ActivityAttachme...Additional ...

User name or ID
Sudarsanan Rengarajan

Organizational Unit
Health Care Business (HCB)

Request Preference
ASAP

Environment
Internal

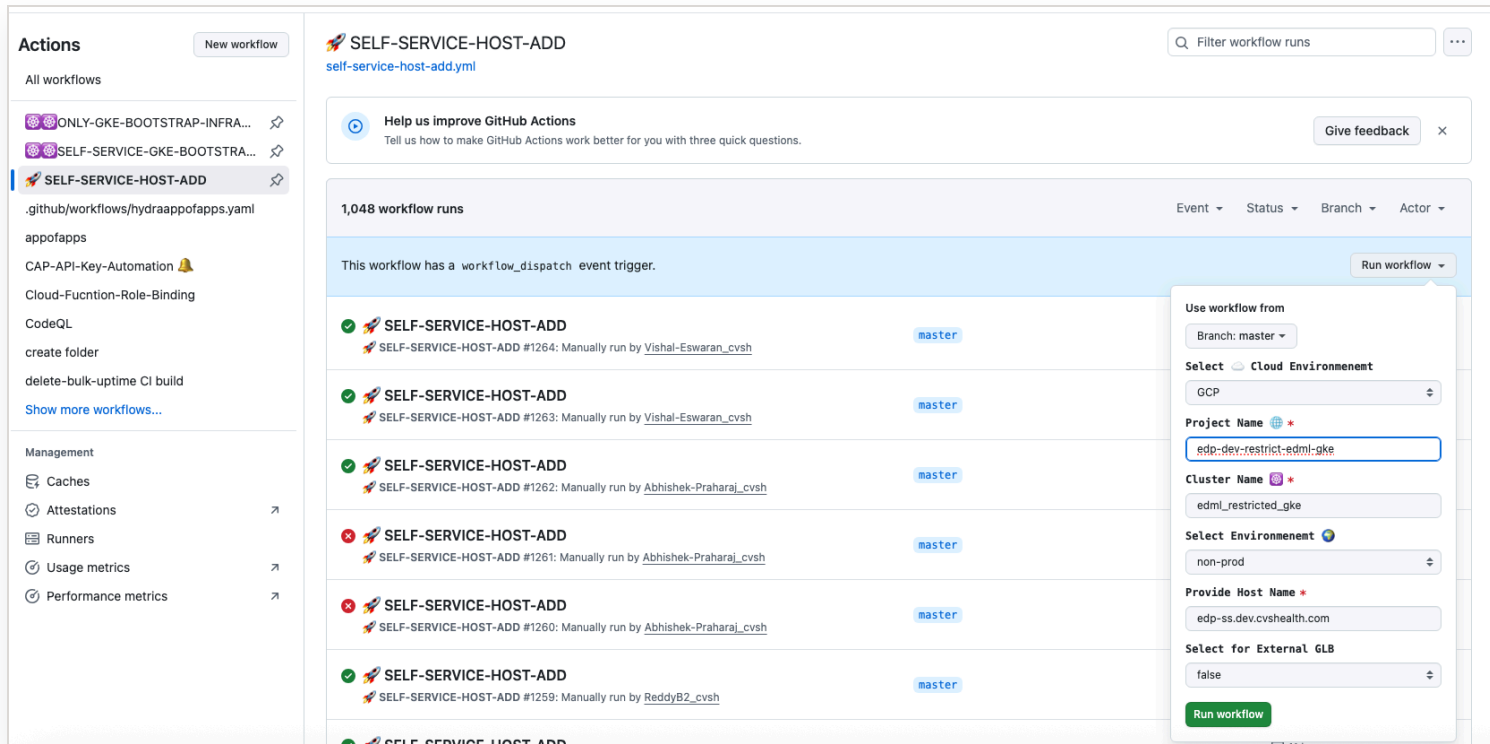
DNS Request Details
Click to view

Automation DNS Request Details
Click to view

Additional Information / Special Instructions

This DNS is for a edp shared services application deployed in shared "edml_restricted_gke" GKE cluster in the namespace "scratch-two-dev", located in GCP project "edp-dev-restrict-edml-gke"

8. From the **Actions** tab of the cluster-specific repository [↗](#), run the  **SELF-SERVICE-HOST-ADD** workflow.

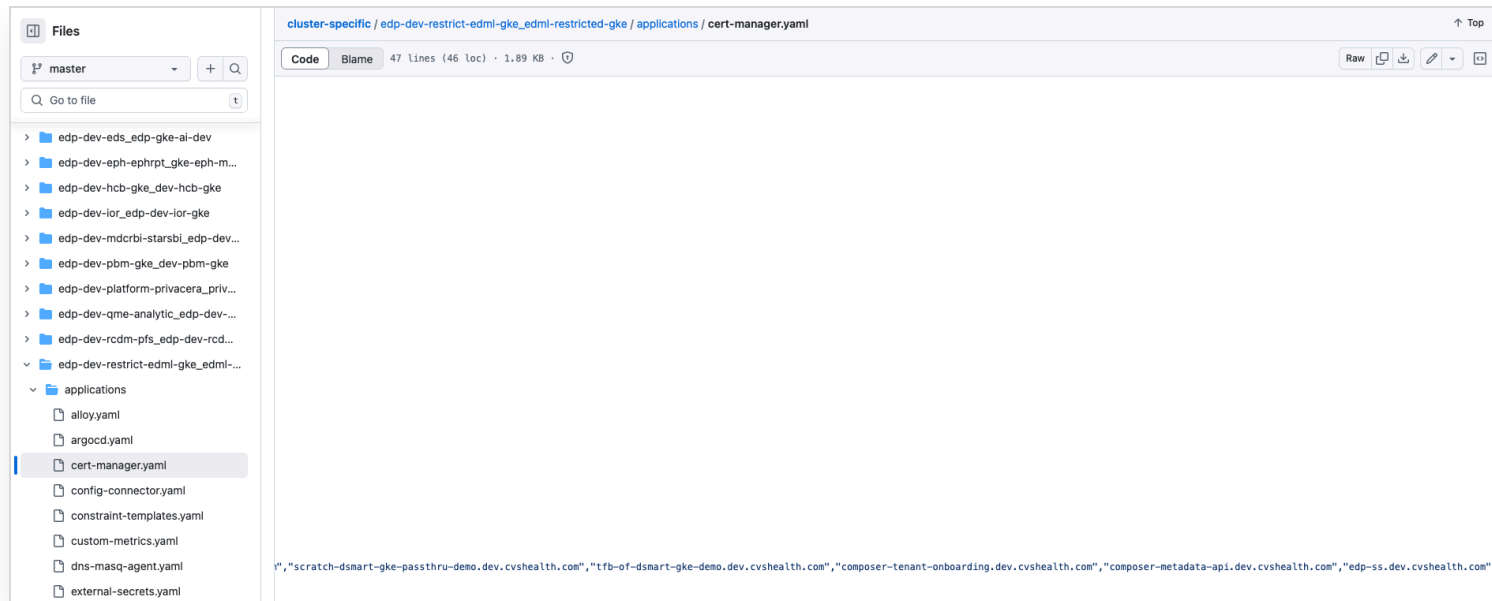


The screenshot shows the GitHub Actions interface for the **SELF-SERVICE-HOST-ADD** workflow. The left sidebar lists various workflows, with **SELF-SERVICE-HOST-ADD** selected. The main area displays the workflow details, including a list of 1,048 workflow runs. A modal dialog is open, showing configuration options for running the workflow:

- Use workflow from: Branch: master
- Select: Cloud Environment: GCP
- Project Name: edp-dev-restrict-edml-gke
- Cluster Name: edml_restricted_gke
- Select Environment: non-prod
- Provide Host Name: edp-ss.dev.cvshealth.com
- Select for External GLB: false

A **Run workflow** button is visible at the bottom of the modal.

From the cluster-specific [↗](#) repository, the **applications/cert-manager.yaml** file for your cluster updates to include your AppURL.



Your application namespace is created. The **applications** directory includes a default service account to authenticate your application (e.g., **scratch-two-dev-ns-serviceaccount.yaml**).

Set up Kubernetes Service Account (KSA)

You must set up and configure a KSA to enhance security by reducing the security risk associated with long-lived credentials. Then, you must configure the KSA with Workload Identity Federation (WIF), which is a Google Cloud feature that allows Kubernetes workloads to authenticate to Google Cloud services without needing to manage service account keys.

1. Set up your KSA. You can:

- Use the default service account to deploy your workflows.
- Create and configure a non-default KSA from your namespace.

2. From [PRIME > Compute Services](#) , select **GKE Workload Identity** to configure the KSA with WIF.
For more: [PRIME > Implement the service](#).

Repository deployment setup

From the application repository **Settings** tab, the repo administrator must configure the repository with the cluster credentials and required Developer Experience (DevEx) properties to ensure execution pipelines do not block application deployment.

For more on required DevEx properties: [SharePoint > Application Mapping - Custom Properties](#) .

1. From the **Code and automation** section, select **Environments > New environment** to create a new environment with your namespace name (e.g., **scratch-two-dev**).

For more: [GitHub Docs > Creating an environment](#) .

The screenshot shows the GitHub repository settings for 'gha-java-web-maven-scratch' in the 'cvsh-health-source-code' organization. The 'Environments' tab is selected, displaying a list of environments: 'scratch-two-prod', 'scratch-two-pdev', and 'scratch-two-dev'. The 'scratch-two-dev' environment is highlighted with a red box and indicates it has '3 variables'. The left sidebar contains navigation links for General, Access, Collaborators and teams, Team and member roles, Code and automation, Branches, Tags, Rules, Actions, Models, Webhooks, Copilot, and Environments.

2. From the **Environment variables** section of the **Environments / Configure** screen, enter names and values for the following variables:

1. **CLUSTER_NAME**: Your cluster name from the **GKE Cluster Name** column of [Confluence > GKE Shared Cluster Details](#) .
2. **CLUSTER_TYPE**: GKE.
3. **MANIFESTS_REPO_NAME**: Your manifest repo name from the **Manifest Repo** column of [Confluence > GKE Shared Cluster Details](#) .

Environment variables

[Add environment variable](#)

Variables are used for non-sensitive configuration data. They are accessible only by GitHub Actions in the context of this environment by using the [variable context](#).

Name 	Value	Last updated		
 CLUSTER_NAME	edml-restricted-gke	3 months ago		
 CLUSTER_TYPE	GKE	4 months ago		
 MANIFESTS_REPO_NAME	edml_restricted_gke-gke_manifests	2 months ago		

For more: [GitHub Docs > Creating configuration variables for an environment](#) .

- From the **Security > Secrets and variables > Actions** section, follow the **<CLUSTER_NAME>_ARGOPASSWORD** name standard to create a new **Repository secret** (e.g., **EDML_RESTRICTED_GKE_ARGOPASSWORD**).

For more: [GitHub Docs > Creating secrets for a repository](#) .

- In the **Secret** field enter the administrator password for Argo CD and click **Add secret**.

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets [New repository secret](#)

Name	Last updated
EDML_RESTRICTED_GKE_ARGOPASSWORD	4 months ago

Organization secrets

Name	Last updated
APOLLO_KEY	4 months ago
ARGOPASS	2 years ago
ARTIFACTORY_DEVSECOPS_ATTEST_TOKEN	2 months ago
ARTIFACTORY_DEVSECOPS_ATTEST_USER	8 months ago
ARTIFACTORY_JAVA_SETTINGS_XML_TEMPLATE	2 months ago
ARTIFACTORY_PASSWORD	7 months ago
ARTIFACTORY_TOKEN	2 months ago

The **Repository secrets** section of the **Actions secrets and variables** screen shows the new secret.

- From the **Security > Secrets and variables > Actions > Variables** section, follow the **<CLUSTER_NAME>_ARGOURL** and **<CLUSTER_NAME>_ARGOUSER** name standard to create new **Repository variables** (e.g., **EDM_RESTRICTED_GKE_ARGOURL** and **EDM_RESTRICTED_GKE_ARGOUSER**).

From the **Value** field for **<CLUSTER_NAME>_ARGOURL**, enter the URL from the **ArgoURL** column of [Confluence > GKE Shared Cluster Details](#) for your cluster.

For more: [GitHub Docs > Creating configuration variables for a repository](#).

General

Access

- Collaborators and teams
- Team and member roles

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models
- Webhooks
- Copilot
- Environments
- Pages
- Custom properties

Security

- Advanced Security
- Deploy keys
- Secrets and variables
- Actions
- Codespaces
- Dependabot

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment variables

Manage environment variables

Name	Value	Environment	Last updated
CLUSTER_NAME	edml-restricted-gke	scratch-dev	3 months ago
CLUSTER_TYPE	GKE	scratch-dev	4 months ago
MANIFESTS_REPO_NAME	edml_restricted_gke-gke_mani...	scratch-dev	2 months ago

Repository variables

New repository variable

Name	Value	Last updated
EDML_RESTRICTED_GKE_ARGOURL	internal-argocd-non-prod.edml-restricted...	4 months ago
EDML_RESTRICTED_GKE_ARGOUSER	admin	4 months ago

Organization variables

Name	Value	Last updated
ARTIFACTORY_DOCKER_NAMESPACE	cvs-enterprise-it-docker-local	8 months ago

The **Repository variables** section of the **Actions secrets and variables** screen shows the new variables.

The repository is configured for cluster deployment.

Repository CI/CD setup

You must add Continuous Integration Continuous Deployment (CI/CD) workflow configurations to your application repository, so you can automate testing and deployment processes, ensure faster delivery of

high-quality software, and improve collaboration among team members.

For projects with CI/CD setup examples:

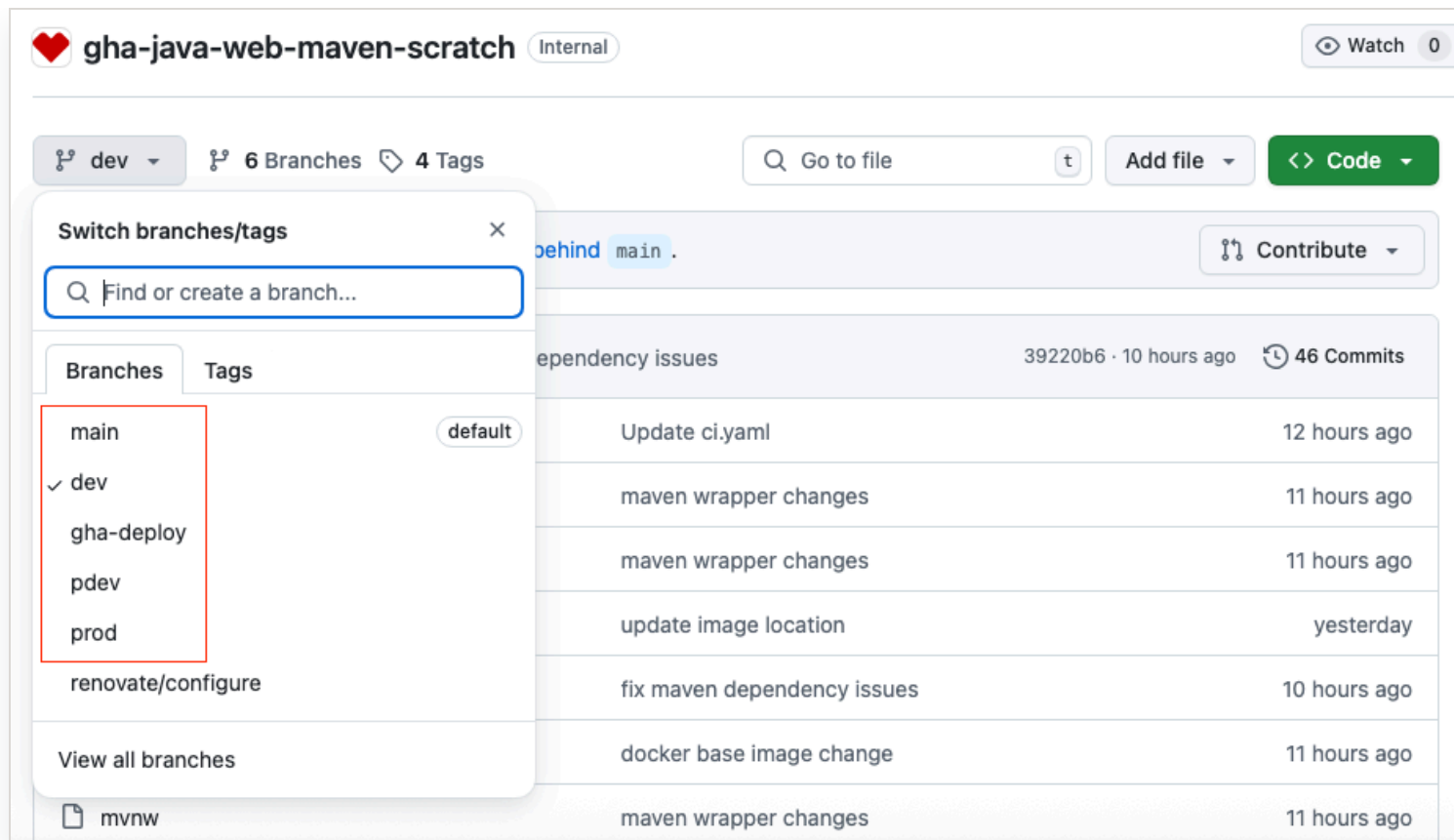
- [gha-java-web-maven-scratch repository](#) 
- [scratch-340b-dsmart-demo repository](#) 
- [rxomni-drug-search repository](#) 

CI configuration

From the **.github/workflows** directory of your repository, you must add a **ci.yaml** file to automatically test code changes as you merge them, ensuring early issue detection.

For more: [CVS DevEx, CICD Customer Documents > Step by Step Guide for CI Workflow Setup](#)  .

1. Create branches for each of your environments (e.g., **dev**, **gha-deploy**, **pdev**, **prod**). For more: [GitHub Docs > Creating a branch](#)  .



2. From the root directory, create a **.github/workflows** subdirectory.

3. From the **.github/workflows** directory, create a **ci.yaml** file.

From the **uses** attribute, append `@latest` to ensure you use the latest DevEx workflow. From the [gha_workflow_actions/.github/workflows repository](#), you can see all of the reusable workflows including the **maven_docker.yaml** workflow for this example.

```
name: Run Docker Pipeline
on: [workflow_dispatch]
jobs:
```

```
docker_build:
  uses: cvs-health-source-
code/gha_workflow_actions/.github/workflows/maven_docker.yaml@latest
  with:
    JAVA_VERSION: "20"
    JAVA_BUILD_COMMAND: "mvn clean package"
    SAST_TOOL: snyk
    SCA_TOOL: snyk
  secrets: inherit
```

JFrog

You must publish a Docker image of your repository to a JFrog repository to deploy your application.

You must have access to JFrog. For more: [Confluence > DevEx Jfrog Artifactory Access and Life Cycle Policy](#) .

1. From the **Actions** tab, select **Run Docker Pipeline > Run workflow > Run workflow**.

The screenshot shows the GitHub interface for the repository `cvs-health-source-code`. The `Actions` tab is selected, and the `Run Docker Pipeline` workflow is highlighted in the left sidebar. The workflow run details show a single run with the status `Success`. A `Run workflow` button is visible in the workflow run details.

The workflow completes the CI stages and pushes your application to the JFrog registry. (e.g., cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch).

The screenshot shows the JFrog Platform interface. The `Artifacts` section is selected, and the `gha-java-web-maven-scratch` repository is chosen. The artifact `39220b62df4fb75f9e34f08bbfd81ad727b02095` is highlighted in the list. The artifact details show the `Name`, `Repository Path`, `File URL`, and `Created` timestamp.

2. From the **Docker Build** section of the action logs, identify the image version.

docker_build / Docker Build
succeeded 1 hour ago in 4m 1s

Jobs

- ✓ docker_build
 - ✓ Setup Pipeline
 - ✓ Build
 - ✓ SAST Scan
 - ✓ SCA Security Scan
 - ✓ Test
 - ✓ Lint
 - ✓ Release Version
 - Push Artifact
 - ✓ Docker Build
 - ✓ Container Scan
 - Push Artifact Gated
 - Docker Publish
 - ✓ Docker Publish Gated
 - ✓ Generate SBOM
 - Update Harness Values

1308
1309 #11 [7/7] WORKDIR /home/edpssuser/gha-java-web-maven-scratch
1310 #11 DONE 0.1s
1311
1312 #12 exporting to image
1313 #12 exporting layers
1314 #12 exporting layers 0.2s done
1315 #12 writing image sha256:d8a3f78a22d25705da6638aff2b57fe431662670b3e1019e61c76b2f6bd9d75e done
1316 #12 naming to cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch/gha-java-web-maven-scratch:39220b62df4fb75f9e34f08bbfd81ad727b02095 done
1317 #12 DONE 0.2s
1318
1319 1 warning found (use docker --debug to expand):
1320 - LegacyKeyFormat: "LABEL key=value" should be used instead of legacy "LABEL key value" format (line 5)
1321 DOCKER_SAVE_COMMAND=docker save -o /home/runner/_work/gha-java-web-maven-scratch/gha-java-web-maven-scratch/image.tar cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch/gha-java-web-maven-scratch:39220b62df4fb75f9e34f08bbfd81ad727b02095
1322 cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch/gha-java-web-maven-scratch:39220b62df4fb75f9e34f08bbfd81ad727b02095
1323 Image info start
1324 cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch/gha-java-web-maven-scratch:39220b62df4fb75f9e34f08bbfd81ad727b02095cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch/gha-java-web-maven-scratch:39220b62df4fb75f9e34f08bbfd81ad727b02095Image info end
1325 ▶ Run actions/cache/save@v4
1437 /usr/bin/tar --posix -cf cache.tgz --exclude cache.tgz -P -C /home/runner/_work/gha-java-web-maven-scratch/gha-java-web-maven-scratch --files-from manifest.txt -z
1438 Sent 0 of 247521888 (0.0%), 0.0 MBs/sec
1439 Sent 0 of 247521888 (0.0%), 0.0 MBs/sec

Snyk CI setup

You must integrate Snyk into your CI pipeline via the reusable action template to automatically detect security vulnerabilities. For more [SharePoint > Snyk CI \(GitHub Action\) Setup](#) .

CD configuration

From your application repository, you must create a deployment configuration branch to automatically release code to production, enabling faster and more reliable updates.

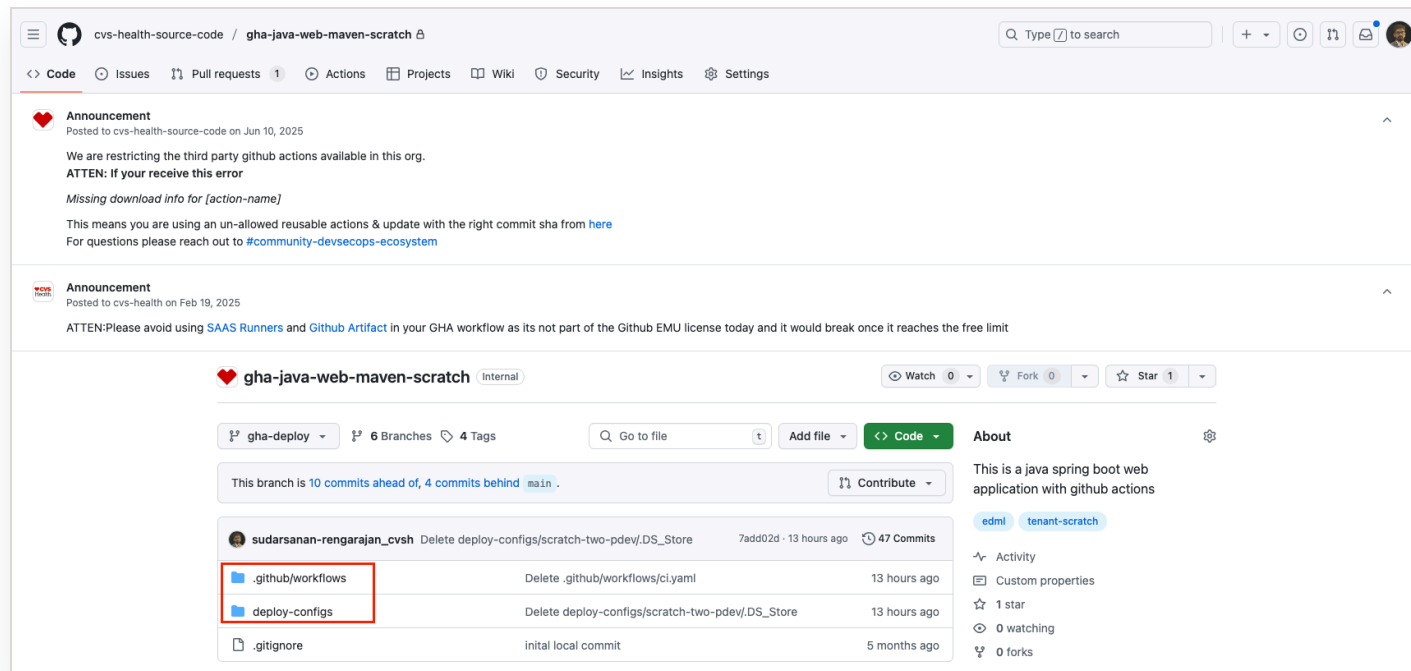
For more: [CVS DevEx, CICD Customer Documents > Gitops Kubernetes Deployment Introduction](#) .

1. Create a **gha-deploy** branch. For more: [GitHub Docs > Creating a branch](#) .

2. From the root directory, create two subdirectories:

- **.github/workflows**

o deploy-configs



3. From the **.github/workflows** folder, add a **cd.yml** file with the following contents:

```
name: App Deployment
on:
  workflow_dispatch:
  push:
    branches: [gha-deploy]
    paths: ['**/values/v1.yaml']
permissions:
  id-token: write
  contents: write
  pull-requests: write
  jobs:
  detect-changes:
    runs-on:
```

```

group: cvs-linux-self-hosted
outputs:
  CHANGED_ENVIRONMENTS: ${ steps.set-matrix.outputs.CHANGED_ENVIRONMENTS }
steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0
  - name: Detect changes
    id: changes
    uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
    with:
      base: ${ github.ref_name }
      filters: |
        scratch-two-dev_v1: '**/deploy-configs/scratch-two-dev/values/v1.yaml'
        scratch-two-pdev_v1: '**/deploy-configs/scratch-two-pdev/values/v1.yaml'
        scratch-two-prod_v1: '**/deploy-configs/scratch-two-prod/values/v1.yaml'
  - name: Build Environment Matrix
    id: set-matrix
    run: |
      ENVS=()
      if [ "${ steps.changes.outputs.scratch-two-dev_v1 }" = "true" ]; then
        ENVS+=("{\"ENV_NAME\":\"scratch-two-dev\",\"HELM_FILE_VERSION\":\"v1\"}")
      fi
      if [ "${ steps.changes.outputs.scratch-two-pdev_v1 }" = "true" ]; then
        ENVS+=("{\"ENV_NAME\":\"scratch-two-pdev\",\"HELM_FILE_VERSION\":\"v1\"}")
      fi
      if [ "${ steps.changes.outputs.scratch-two-prod_v1 }" = "true" ]; then
        ENVS+=("{\"ENV_NAME\":\"scratch-two-prod\",\"HELM_FILE_VERSION\":\"v1\"}")
      fi
      if [ ${#ENVS[@]} -gt 0 ]; then
        JSON=$(IFS=,; echo "${ENVS[*]}")
        echo "CHANGED_ENVIRONMENTS=$JSON" >> $GITHUB_OUTPUT
      else
        echo "CHANGED_ENVIRONMENTS=[]" >> $GITHUB_OUTPUT
      fi
  deploy:

```

```
uses: cvs-health-source-code/gitops-cd-workflow/.github/workflows/helm-
deploy.yaml@v1
needs: detect-changes
if: needs.detect-changes.outputs.CHANGED_ENVIRONMENTS != '[]'
strategy:
  fail-fast: false
  matrix:
    environment: ${ fromJson(needs.detect-changes.outputs.CHANGED_ENVIRONMENTS) }
secrets: inherit
with:
  app-name: "gha-java-web-maven-scratch"
  environment: ${ matrix.environment.ENV_NAME }
  helm-template-repo: ${ vars.HELM_TEMPLATE_REPO }
  helm-file-version: ${ matrix.environment.HELM_FILE_VERSION }
  helm-template-branch: main
  canary-weight: "0"
  lob: "account"
  change-description: "EDP SS demo application change for production"
  rfc-short-desc: "This is to deploy EDP SS demo application to production"
  change-domain: "Enterprise"
  change-impacted-website: "CVS.COM"
```

4. From the **deploy-configs** folder, add a **common** directory and follow the - name standard to create namespace directories (e.g., **scratch-two-dev/values**, **scratch-two-pdev/values**, and **scratch-two-prod/values**).

The screenshot shows the GitHub interface for the repository 'gha-java-web-maven-scratch'. The left sidebar displays the file tree with the 'deploy-configs' folder selected. The main content area shows the commit history for the 'deploy-configs' directory. A red box highlights the 'common' folder in the commit history table.

Name	Last commit message	Last commit date
..		
common	Update common.yaml	3 months ago
scratch-two-dev/values	Delete deploy-configs/scratch-two-dev/DS_Store	13 hours ago
scratch-two-pdev/values	Delete deploy-configs/scratch-two-pdev/DS_Store	13 hours ago
scratch-two-prod/values	cd related changes	13 hours ago

- From the **deploy-configs/common** folder, add a **common.yaml** file with configuration specification values for all application deployments.

The screenshot shows a GitHub repository interface. On the left, the file explorer shows the directory structure: `gha-deploy` (selected), `deploy-configs`, `common` (selected), and `common.yaml` (selected). The main area displays the content of `common.yaml`, which is a YAML file for Istio configuration. The file is 35 lines long, 30 loc, and 843 Bytes. It was updated by `sudarsanan-rengarajan_cvsh` 41afda6 · 3 months ago. The configuration includes app-related data, liveness and readiness probe details, startup probe details, and an Istio HTTP route.

```

1 #app related data
2 app:
3   name: gha-java-web-maven-scratch
4   port: 8080
5   gitRepoName: cvs-health-source-code/gha-java-web-maven-scratch
6
7
8 #livenessProbe details
9 livenessProbeDetails:
10   probePath: "/edp/actuator/health"
11   initialDelaySeconds: 30
12
13 #readinessProbe details
14 readinessProbeDetails:
15   probePath: "/edp/actuator/health"
16   initialDelaySeconds: 30
17
18 #startupProbe details
19 startupProbeDetails:
20   probePath: "/edp/actuator/health"
21   initialDelaySeconds: 300
22   periodSeconds: 100
23   failureThreshold: 40
24
25 istioHttp: |
26   - match:
27     - uri:
28       prefix: /edp/welcome
29     route:
30     - destination:
31       host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace }}.svc.cluster.local
32       port:
33         number: {{ .Values.app.port }}
34         subset: {{ regexReplaceAll "\\W+" .Values.istio.currentAppVersion "-" }}
35       weight: 100

```

- From each namespace directory, add a **v1.yaml** file with application-specific configuration specification values.

The screenshot displays the ArgoCD web interface. On the left, the 'Files' sidebar shows a directory structure: `.github`, `deploy-configs`, `common` (containing `common.yaml`), `scratch-two-dev/values` (containing `v1.yaml`), `scratch-two-pdev/values`, and `scratch-two-prod/values` (containing `.gitignore`). The `v1.yaml` file is selected. The main area shows the details for the `gha-java-web-maven-scratch` application, specifically the `deploy-configs / scratch-two-dev / values / v1.yaml` file. The file is owned by `sudarsanan-rengarajan_cvsh` and has been updated. The file content is displayed in a code editor with line numbers 1 through 35. The YAML content defines Kubernetes deployment parameters including namespace, image details, hpa settings, resource requests/limits, and environment variables.

```

1 #namespace, app version and image details
2 skipSecurityContext: true
3 namespaceWorkloadIdentity:
4   serviceAccountName: default
5 appData:
6   nameSpace: scratch-two-dev
7   dockerHub: cvsh.jfrog.io
8   dockerImageGroupName: cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch
9   dockerImageName: gha-java-web-maven-scratch
10  dockerImageTag: 1.0.2
11
12 #hpa settings - min and max replicas and minAvailable for PDB
13 hpaSettings:
14   minAvailable: 1
15   minReplicas: 1
16   maxReplicas: 1
17   averageUtilization: 80
18
19 # Pod resource requests and limits
20 resources:
21   requests:
22     cpu: "200m"
23     memory: "350Mi"
24   limits:
25     cpu: "500m"
26     memory: "1Gi"
27
28 # environment variables for app - secret values come from vault
29 envVars: |
30   - name: TEST_KEY
31     value: TEST_VALUE
32   - name: ENABLE_TRACING
33     value: 'true'
34   - name: ENV
35     value: dev

```

7. To deploy the latest version of your application, from the **v1.yaml** file, update the `dockerImageTag` variable with the latest image tag from the CI workflow.

From the ArgoCD endpoint for your cluster (i.e., [https://internal-argocd-prod.\\$CLUSTER_NAME.cvshealth.com](https://internal-argocd-prod.$CLUSTER_NAME.cvshealth.com)), you can confirm that your application deployed. For more: [Confluence > ArgoCD App of Apps](#) .

In this article

[Video introduction](#)

[Manifest repositories](#)

[Create your namespace](#)

[Set up Kubernetes Service Account \(KSA\)](#)

[Repository deployment setup](#)

[Repository CI/CD setup](#)